Guarded Process Trees: Translation to Petri Nets and Application in Process Mining

Ido Shapira and Gera Weiss
Department of Computer Science
Ben Gurion University of The Negev
Beer Sheva, Israel
ids,geraw@post.bgu.ac.il

Abstract—Process mining extracts models from event logs to provide insights into workflows. Current techniques often struggle to capture counting-based dependencies between activities. We address this challenge by introducing guarded process trees — an enhancement of traditional process trees with guards that control transitions based on activity occurrence counts. We present a two-step method for translating guarded process trees to Petri nets: first converting to a guarded Petri net, then removing the guards while preserving the counting semantics. We formally define and prove the correctness of both translation steps. To demonstrate practical applicability, we develop a method for learning guarded process trees from event logs and implement it using grammatical evolution. Through experiments on both synthetic and real-world datasets, we show that our approach significantly improves precision compared to existing techniques while maintaining model interpretability. The evaluation demonstrates average precision improvements of 4.94-11.14% across data sets and different mining algorithms when enhanced with guards.

I. Introduction

Process mining [1] is a rapidly evolving field focused on extracting process models from event logs [2, 3]. These models provide valuable insights into the underlying processes, enabling organizations to document, analyze, and optimize workflows.

In this paper we propose a novel approach to process mining that addresses the challenge of capturing counting-based dependencies between activities. Traditional process mining techniques often struggle to accurately represent such dependencies, leading to incomplete or inaccurate models [4, 5]. This limitation is particularly evident in scenarios where the number of occurrences of certain activities influences the behavior of the process.

Consider, for example, a queuing system with multiple service stations, each maintaining its own queue. Customers arrive and choose to join the shortest available queue. The decision-making process of each customer depends on the current length of each queue—a dynamic attribute that changes as customers arrive and are served. Modeling such a system requires capturing the dependency between the customer's choice and the current state (i.e., the count of customers) of each queue. Traditional process mining approaches, which often rely on the sequence of events without considering the underlying state of the system, may fail to accurately represent this behavior. They might model the customer's choice as a random selection among queues, ignoring the influence of queue lengths. This oversight can lead to models that do not reflect the actual decision-making process within the system.

To address this, our approach integrates counting-based dependencies into the process model. By incorporating mechanisms to track and utilize the counts of specific activities (e.g., the number of customers in each queue), we can more accurately model systems where such counts influence future behavior. This enhancement allows for the discovery of more precise and informative process models, particularly in contexts where decisions are contingent upon quantitative aspects of the process state.

We use process tree models that have gained popularity due to their structured and hierarchical nature, which facilitates learning from event logs [2]. Guarded process trees, which we propose in this paper, extend traditional process trees by incorporating guards on the edges. These guards are Boolean functions that determine whether a transition can be taken based on the number of times each activity has occurred in the past. This extension allows for the representation of intricate dependencies between activities, enabling the direct modeling of processes that involve counting.

Figure 1 depicts a guarded process tree modeling a queuing system, where eA, lA, eB, and lB denote the actions of entering and leaving queues A and B, respectively. The model comprises five parallel customer instances entering the system, with guards enforcing that each customer selects the currently shortest queue.

We propose a systematic translation from guarded process trees to Petri Nets [6], specifically using the Integer Petri Nets formalism [7]. Our translation consists of two key steps: first, we transform the guarded process tree into an intermediate representation called a Guarded Petri Net, which preserves the guard semantics while providing a bridge to standard Petri net notation. Second, we eliminate the guards by encoding their constraints directly into the Petri net structure through additional places and connections. This approach allows us to leverage existing analysis techniques and tools developed for Petri nets while maintaining the expressive power of guards,

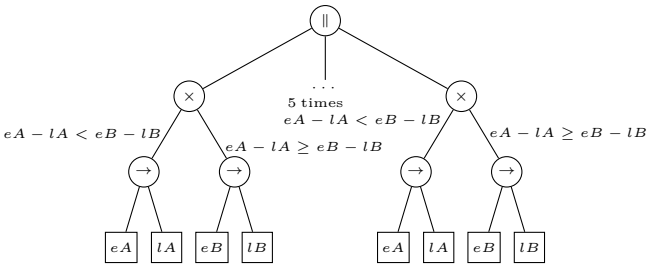


Figure 1: A graphical description of a guarded process tree representing a queuing system where eA, lA, eB, lB represent entering and leaving queues A and B, respectively. The model involves 5 parallel instances where customers enter the queuing system, and the guards ensure that each customer enters the shorter queue.

resulting in models that can be readily compared with and integrated into existing process mining frameworks.

Our translation from guarded process trees to guarded Petri nets follows a compositional approach, preserving the hierarchical structure of the process tree while incorporating guards. For each operator in the process tree (sequence, choice, parallel, and loop), we define corresponding Petri net patterns that capture both the control flow and the guard conditions. The translation maintains the original guards on transitions, ensuring that the counting semantics are preserved. This first translation step results in a guarded Petri net that directly reflects the structure and behavior of the original process tree.

The second translation step, from guarded Petri nets to standard Petri nets, eliminates guards by encoding them into the structure of the net itself. For each guard that counts activity occurrences, we introduce additional places and connections that track these counts using tokens. The translation converts each guarded transition into an unguarded equivalent by adding control places that accumulate tokens based on activity executions. This structural encoding ensures that transitions fire only when their original guard conditions would have been satisfied, effectively preserving the counting semantics without requiring explicit guards.

We formally define the translations and provide illustrative examples to demonstrate the process. We also prove that the translation preserves the semantics of the guarded process tree, ensuring that the traces fitting the guarded process tree are identical to those fitting the translated Petri net [8].

To demonstrate the practical advantages of our translation, we developed a method for learning guarded process trees from event logs. This method first discovers a base process tree using traditional mining techniques, then enhances it by learning guards through grammatical evolution. We integrate the learned guards using our translation framework, enabling a seamless transition from guarded process trees to executable Petri net models.

To validate our approach, we conducted extensive experiments comparing our enhanced models with existing process mining techniques across both synthetic and real-world datasets. Our technique can be applied to any process mining algorithm that produces a process tree, such as the Heuristic Miner [9], Inductive Miner [10], and ILP Miner [11].

We evaluate our approach on synthetic and real-world datasets, including the BPI Challenge 2012 and 2013 datasets, which contain complex event logs with various counting dependencies. The results are compelling: our approach consistently improves model precision while maintaining interpretability. Specifically, we observe significant precision improvements across different mining algorithms - Heuristic Miner shows an average improvement of 4.94%, Inductive Miner demonstrates an 11.14% increase, and ILP Miner achieves a 9.89% enhancement. These improvements are particularly pronounced when capturing counting-based dependencies and complex behavioral patterns that traditional process models struggle to represent accurately. This demonstrates that our translation framework not only preserves the semantic richness of guarded process trees but also enhances the practical utility of process mining in real-world applications.

II. Related Work

Our work builds upon existing research in guarded automata and Petri nets, particularly their extensions and applications in process mining. We consider advancements in process discovery techniques that address challenges related to noise and numerical dependencies [4, 5] in event logs, positioning our contribution within several interconnected research areas.

Guarded automata represent a fundamental concept in our work, incorporating Boolean conditions on transitions to control state changes. These guards typically depend on variables tracking numerical constraints, such as activity occurrence counters. Research by [12] introduced a framework for automata with constraints, while [13] demonstrated how numerical constraints enhance traditional automata models' expressiveness. These works showcase guarded systems' capability to capture complex dependencies between activities, a property we leverage in our guarded process trees.

In the domain of Petri nets, which have long been foundational for modeling concurrent systems, various extensions have enhanced their expressiveness. Notable developments include inhibitor and reset arcs for complex token flow constraints [6], and colored Petri nets (CPNs) [14] that associate tokens with data values. Integer Petri Nets, allowing negative token counts [7], provide additional flexibility for modeling numerical dependencies, as demonstrated by [15]. We adopt this formalism in our translation process to preserve numerical constraints.

Process discovery techniques, particularly the Heuristic Miner [9], have advanced the field by considering

frequency-based dependencies between activities. While the Heuristic Miner improves upon simpler algorithms by incorporating numerical heuristics for noise filtering, it has limitations in capturing fine-grained numerical constraints. Our work extends these ideas by explicitly incorporating numerical dependencies within a formal model.

Declarative process models, exemplified by the Declare framework [16], offer an alternative approach by specifying constraints rather than fixed sequences. Our work complements these models by integrating numerical constraints directly into the process model. Recent research has explored various constraint types in process mining, from temporal constraints [17] to data constraints [18]. Our approach builds on these ideas by introducing guards for numerical dependencies.

The practical implications of our work extend to business process management (BPM), where capturing complex dependencies and numerical constraints can enhance model accuracy. This enhancement leads to more effective process analysis, optimization, and compliance checking, as demonstrated by [19] and [20]. Through the integration of guards and their translation to Petri nets, our approach contributes to the evolving landscape of process mining and BPM tools.

III. Preliminaries

This paper uses a variant of Petri Nets called Integer Petri Nets [7] where places may contain a negative number of tokens and edges exiting transitions can be labeled with negative values. See also Kansal's work on Signed Petri Net [15].

Definition 1. An (Integer) Petri net is a tuple $PN = (PLC, TRN, \rightarrow)$ where:

- $PLC = \{(P_1, m_1), \ldots, (P_n, m_n)\}$ is a set of places with names $P = \{P_1, \ldots, P_n\}$ and the corresponding number of initial tokens in each of them.
- $TRN = \{(l_1, x_1), \ldots, (l_m, x_m)\}$ is a set of transitions with names $L = \{l_1, \ldots, l_m\}$ and symbols, $x_i \in \Sigma$,
- $\rightarrow \subseteq (P \times \mathbb{Z} \times T) \cup (T \times \mathbb{Z} \times P)$ is the weighted flow relation.

Throughout this paper, we use "Petri Nets" as a shorthand for "Integer Petri Nets."

A marking represents the distribution of tokens across the places in a Petri net at a given moment:

Definition 2. A marking of a Petri net $PN = (PLC, TRN, \rightarrow)$ is a function $M \colon PLC \rightarrow \mathbb{Z}$ that assigns to each place an integer representing the number of tokens in that place. A marking M is initial if for each $(P_i, m_i) \in PLC$, $M(P_i) = m_i$.

A run of a Petri net represents a valid execution sequence, where transitions fire according to the token distribution and flow relation. A trace of a Petri net is the sequence of transitions fired during a run:

Definition 3. A sequence $(M_0, t_0, \cdots, t_{n-1}, M_n)$ of alternated markings and transitions is a run of a Petri net $PN = (PLC, TRN, \rightarrow)$ if:

1) M_0 is an initial marking of PN.
2) For each $r = 0, \ldots, n-1$ and for every place p in P:
   - If $(p, w, t_r) \in \rightarrow$, then $M_r(p) \geq w$.
   - $M_{r+1}(p) = M_r(p) - w^- + w^+$ where
$$w^- = \begin{cases} w, & \text{if } (p, w, t_r) \in \rightarrow, \\ 0, & \text{otherwise} \end{cases},$$
$$w^+ = \begin{cases} w, & \text{if } (t_r, w, p) \in \rightarrow, \\ 0, & \text{otherwise} \end{cases}.$$
3) $M_n(SNK(PN)) = 1$.

Assuming that $t_i = (l_i, x_i)$, the trace of this run is $(x_1, \ldots, x_n)$, which is also referred to as a trace of the Petri net.

IV. Guarded Process Trees (GPTs)

Guarded process trees include guards on the edges of process trees, dictating entry into a sub-tree based on past triggered activities. This representation is proposed as a method to capture numerical dependencies within processes.

The following definition specifies the syntax of guarded process trees, which enhance standard process trees with Boolean guards on edges. These guards are predicates over execution history that determine if a sub-tree is active based on past activity occurrences. Formally, guards are functions mapping activity histories Σ^* to Boolean values $\{\text{True}, \text{False}\}$.

Definition 4. Let Σ be a set of activities and let $\tau \notin \Sigma$ represent an unobservable activity. A guarded process tree Q, is defined (recursively) as any of:

- $x \in \Sigma \cup \{\tau\}$ an (observable or non-observable) activity,
- $o((g_1, T_1), \ldots, (g_n, T_n))$, for $o \in \{\times, +, \rightarrow\}$ and $n > 1$,
- $*((g_1, T_1), (g_2, T_2))$;

where $T_1, \ldots, T_n$ are guarded process trees, and $g_i \colon \Sigma^* \rightarrow \{\text{True}, \text{False}\}$ are guards. We assume, for simplicity, that if $T_i = \tau$, then its guard g_i is the constant function $g_i \equiv \text{True}$. Let $\mathcal{G} = \{\text{True}, \text{False}\}^{\Sigma^*}$ be the set of all possible guards.

The semantics of a guarded process tree are defined through its set of valid traces. We devide the definition into two parts: one for guarded traces and one for unguarded traces. A guarded trace is a sequence $t \in (\Sigma \cup \mathcal{G})^*$ where activities and guards are interleaved. An unguarded trace is a sequence $t \in \Sigma^*$ where only activities are present.

The following definition specifies when a guarded trace fits a guarded tree:

Definition 5. A guarded trace $t \in (\Sigma \cup \mathcal{G})^*$ fits a guarded process tree T if one of the following conditions holds:

- If $T = \tau$ and $t = \varepsilon$.

- If $T \in \Sigma$ and $t = T$.
- If $T = \times((g_1, T_1), \ldots, (g_n, T_n))$ and, for some i, $t = g_i t'$ where t' fits T_i.
- If $T = +((g_1, T_1), \ldots, (g_n, T_n))$ and $t \in (\bigwedge_{i \in K} g_i \wedge \bigwedge_{j \notin K} \neg g_j) \circ \|_{i \in K} t_i$ for some $K \subseteq \{1, \ldots, n\}$ and guarded traces such that t_i fits T_i for each $i \in K$. Here, $\|$ denotes the interleaving of traces defined recursively as:

$$(t_1 \| t_2) = \begin{cases} \{t_1\} & \text{if } t_2 = \epsilon; \\ \{t_2\} & \text{if } t_1 = \epsilon; \\ adv_1(t_1, t_2) \cup adv_2(t_1, t_2) & \text{otherwise.} \end{cases}$$

  where

  $adv_1(t_1, t_2) = \{\alpha \circ t \colon t_1 = \alpha \circ t', \alpha \in \mathcal{L}((\mathcal{G} \cup \Sigma)^* \Sigma), t \in t' \| t_2\}$

  and

  $adv_2(t_1, t_2) = \{\alpha \circ t \colon t_2 = \alpha \circ t', \alpha \in \mathcal{L}((\mathcal{G} \cup \Sigma)^* \Sigma), t \in t_1 \| t'\}$.
- If $T = \rightarrow ((g_1, T_1), \ldots, (g_n, T_n))$ and $t = t_1 \circ \cdots \circ t_n$ where, for each i, $t_i = g_i \circ t_i'$ and t_i' fits T_i.
- If $T = *((g_0, T_0), (g_1, T_1))$ and $t = t_0 \circ \cdots \circ t_{2n}$ where, for each i, $t_i = g_{i \bmod 2} \circ t_i'$ and t_i' fits $T_{i \bmod 2}$.

For the following definition, which specifies when an unguarded trace fits a guarded tree, we define the function:

$$\mathrm{dg}(t) = \begin{cases} t & \text{if } t \in \Sigma^*; \\ \mathrm{dg}(t_1) \circ \mathrm{dg}(t_2) & \text{if } t = t_1 \circ g \circ t_2 \text{ where } g \in \mathcal{G}. \end{cases}$$

that drops all the guards from a given guarded trace.

Now, we can define when an unguarded trace fits a guarded tree:

Definition 6. A trace $t \in \Sigma^*$ fits a guarded process tree T if there exists a guarded trace $t' \in (\Sigma \cup \mathcal{G})^*$ that fits T where $t = \mathrm{dg}(t')$ and for every $t_1', t_2' \in (\Sigma \cup \mathcal{G})^*$ and $g \in \mathcal{G}$ such that $t_1' \circ g \circ t_2' = t'$, the guard g evaluates to true on the history: $g(\mathrm{dg}(t_1')) = \text{True}$.

An example of a guarded trace, t', that fits the guarded process tree in Figure 1 is $t' = g_B, eB, g_A, eA, lA, g_A, eA, g_B, eB, g_A, eA, lA, lB, lA, lB$. The corresponding fitted trace, $t = dg(t')$, obtained by removing the guard annotations while preserving the control flow, is $t = eB, eA, lA, eA, eB, eA, lA, lB, lA, lB$ where g_A is $eA{-}lA < eB{-}lB$ and g_B is $eA{-}lA \geq eB{-}lB$.

V. Guarded Petri Nets

This section introduces guarded Petri nets (GPNs), which extend standard Petri nets with Boolean guards on transitions. These guards, similar to those in guarded process trees, determine when transitions can fire based on execution history. We then present a translation from guarded Petri nets to standard Petri nets that preserves the semantics while encoding the guards into the net structure. This approach allows us to leverage existing analysis techniques for standard Petri nets while maintaining the expressiveness of guards.

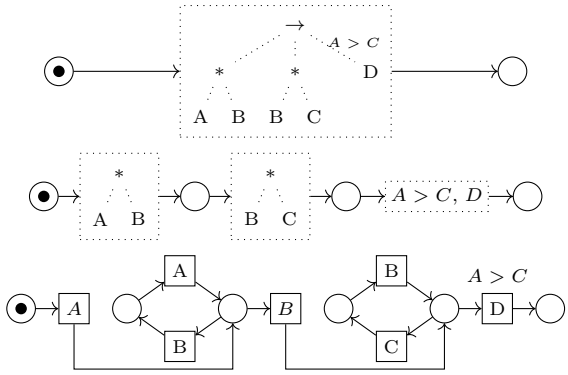


Figure 2: An example of the translation of a guarded tree to a guarded Petri Net. We used rule (d) for the first translation and rules (a) and (c) for the second.

The formal definition of a guarded Petri net, inspired by the work of Jensen [14], is as follows:

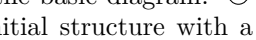
Definition 7. A Guarded-Petri Net (GPN) is a tuple $(PLC, TRN, \rightarrow, G)$ where $(PLC, TRN, \rightarrow)$ is a Petri Net and $G \colon TRN \rightarrow \mathcal{G}$ is a function that assigns guards to transitions.

The semantics of guarded Petri nets is defined, like the semantics of standard Petri nets in terms of their runs:

Definition 8. A sequence $(M_0, t_0, \ldots, M_{n-1}, t_{n-1}, M_n)$ is a run of a GPN $= (PLC, TRN, \rightarrow, G)$ if it is a run of the $PN = (PLC, TRN, \rightarrow)$ such that for each $r = 0, \ldots, n-1$, $G(t_r)(x_0, \ldots, x_{r-1}) = \text{True}$, where $(x_0, \ldots, x_r)$ is the trace of the run.

VI. Translation of Guarded Trees to Petri Nets

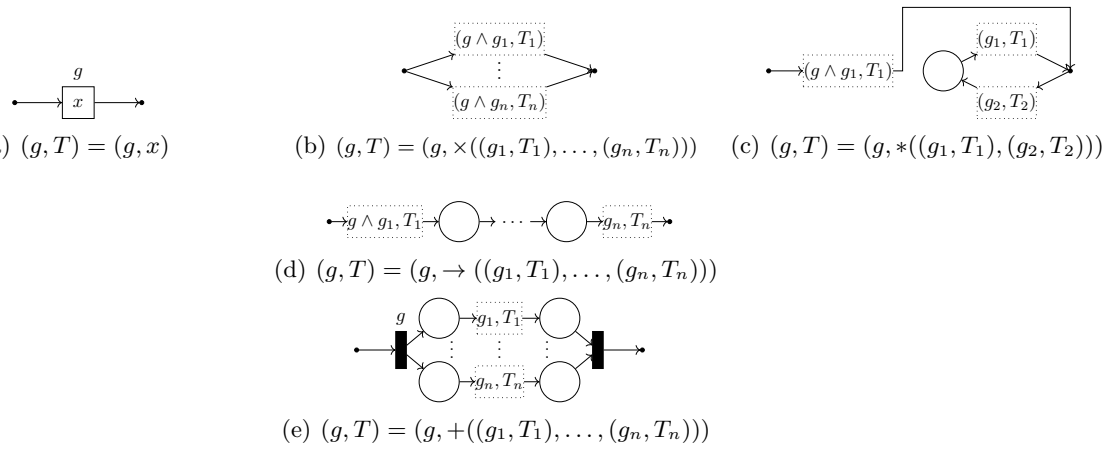
Inspired by the work of Ghahfarokhi et. al [21], we define the translation of a guarded process tree to a guarded Petri net using the diagram shown in Figure 3. The full mathematical specification of this translation moved to an appendix becuase of space limitations.

The translation process using Figure 3 follows a recursive pattern. Given a guarded process tree T, begin with the basic diagram: 

. This represents the initial structure with a source place (containing one token), a placeholder labeled with the tree T, and a sink place. Then, recursively replace each dotted placeholder in the diagram with the appropriate pattern from Figure 3, based on the main operator of the corresponding subtree. This replacement process continues until all dotted placeholders have been expanded into their complete representations.

VII. Translating Guarded Petri Nets to a Petri Nets

We are now ready to propose a translation from guarded Petri nets to unguarded ones. We assume that the guards are given in a specific format for this translation. While in the rest of this paper, we allowed guards to assign truth values to words arbitrarily; we now assume that they do so



(a) $(g, T) = (g, x)$ (b) $(g, T) = (g, \times((g_1, T_1), \ldots, (g_n, T_n)))$ (c) $(g, T) = (g, *((g_1, T_1), (g_2, T_2)))$

(d) $(g, T) = (g, \rightarrow((g_1, T_1), \ldots, (g_n, T_n)))$

(e) $(g, T) = (g, +((g_1, T_1), \ldots, (g_n, T_n)))$

Figure 3: Petri net representations of different task structures.

only by counting the number of occurrences of each letter and then applying a logical expression that compares the counts to constant numbers. Note that this is consistent with all the examples in the paper.

Definition 9 (Guarded Petri Nets to Petri Nets). Let $GPN = (PLC, TRN, \rightarrow, G)$ be a Guarded Petri Net. For each $t = (l, x) \in TRN$, let

$$G(t) = \bigvee_{i=1}^{n_t} \bigwedge_{j=1}^{n_{t,i}} \left(\sum_{k=0}^{n_{t,i,j}} \alpha_{t,i,j,k}\, x_{t,i,j,k} \geq C_{t,i,j} \right)$$

be its guard in disjunctive normal form (DNF), where $\alpha_{t,i,j,k}, C_{t,i,j} \in \mathbb{Z}$ and $x_{t,i,j,k} \in \Sigma$.[1] Define $PN(GPN) = (PLC', TRN', \rightarrow')$ as follows:

1) $PLC' = PLC \cup grdPLC$, where $grdPLC = \{(p_{t,i,j}, -(C_{t,i,j} - 1)) : (l, x) \in TRN,\ i \in [n_t],\ j \in [n_{t,i}]\}$
2) $TRN' = \bigcup_{(l,x) \in TRN} \{((i, l), x) : i \in [n_t]\}$.
3) Copy $s \xrightarrow{\lambda} (l, x)$ and $(l, x) \xrightarrow{\lambda} d$ to $s \xrightarrow{\lambda}{}' ((i, l), x)$ and $((i, l), x) \xrightarrow{\lambda}{}' d$, respectively, for all $i \in [n_t]$.
4) For each $t = (l, x)$, connect $p_{t,i,j} \leftrightarrow' ((i, l), x)$ for all $i \in [n_t], j \in [n_{t,i}]$.
5) For all $x_{t,i,j,k} \in \Sigma$ with $(l', x_{t,i,j,k}) \in TRN'$, add $(l', x_{t,i,j,k}) \xrightarrow{\alpha_{t,i,j,k}}{}' p_{t,i,j}$.

We present an example how a guarded-petri-net is translated to a petri-net.

Example 1 (Daily-Routine). The example describes a routine where you repeatedly eat and go to the gym until you go to sleep. You are allowed to sleep if: (1) You have eaten at least three times and gone to the gym at least once; (2) Your eating-to-gym balance is maintained (you have eaten at least three times as much as you have exercised). This routine enforces a balance, ensuring that you do not just eat all day without working out before heading to bed.

[1] If $G(t) = \text{True}$, then $n_t = 1$ and $n_{t,1} = 0$.

Figure 4 is a guarded-petri-net that model the daily routine in the example. Figure 5 is the petri-net translated from Figure 4. The guard of sleep contains two conjunctions, therefore, the transition $(t3, sleep)$ replaced with two transitions: $((1, t3), sleep)$ and $((2, t3), sleep)$. $((1, t3), sleep)$ could fire only if the first conjunction is true and $((2, t3), sleep)$ only if the second conjunction is true. The first conjunction contains two literals, therefore, we connect $((1, t3), sleep)$ with two new places: $p_{3,1,1}$ and $p_{3,1,2}$. The second conjunction contains only one literal, therefore, we connect $((2, t3), sleep)$ with one new place: $p_{3,2,1}$. Each new place initialized with number of tokens according to his literal, and connected to the transitions according to his literal. A new place contain at least one token only if his literal is true.

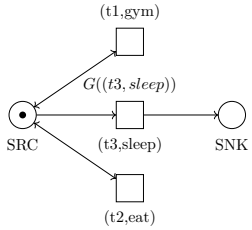


Figure 4: A guarded petri net that model Example 1.

The following proposition, whose proof is moved to an appendix due to space limitations, states the correctness of the above translation:

Proposition 1. A sequence $(x_1, \ldots, x_l)$ is a trace of the Guarded-Petri-Net $GPN = (PLC, TRN, \rightarrow, G)$ if and only if it is a trace of the Petri Net $PN(GPN) = (PLC', TRN', \rightarrow')$.

VIII. Grammatical Evolution for Guard Mining

The missing piece for applying the translations described above to process mining is a method for learning guarded process trees from sets of traces. One practical

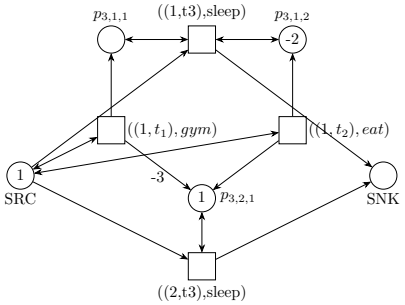


Figure 5: A graphical description of the translated unguarded petri net of the guarded petri net in Figure 4.

approach is first to learn a standard process tree using existing miners or algorithms and then apply grammatical evolution [22] to learn guards on the edges of that tree. These guards capture numerical or logical dependencies in the data, refining the mined model to better reflect actual process constraints. This section introduces our approach for discovering transition guards using grammatical evolution. Our method systematically extracts conditions that dictate transitions enabling based on execution traces. The approach consists of data extraction, grammar definition, and optimization using evolutionary algorithms to derive guards effectively [23]. The goal is to infer guard conditions for transitions in a process model based on execution traces. To achieve this, we analyze transitions enabling conditions and choices at each execution step. Specifically, for each transition, we generate a dataset encoding transition occurrences and avoidance and employ grammatical evolution to identify optimal guard conditions.

Since learning guards may require application-specific insights, we base our mining on user-provided heuristics for a specific guard in this tree that we want to refine. We assume that we are given a function h that takes the following parameters:

- An unguarded process tree (i.e., a guarded process whose guards are all theologies).
- A specific guard in this tree that we want to refine.
- A trace, t, from the system's event log.
- An index,$i \in \{0, |t| - 1\}$, of an activity in this trace.

The function returns two numbers representing:

- A reward/penalty, $rwd(t, i, 1)$,for accepting $t[..i]$.
- A reward/penalty, $rwd(t, i, 0)$,for not-accepting $t[..i]$.

Negative numbers mean penalties, negative mean rewards, and zero means that the guard is not considered at position i of t.

Our approach to guard mining is based on a Backus-Naur Form (BNF) grammar [24] that we defined to encode the syntactic rules for constructing guard expressions, including arithmetic operations, logical conditions, dataset features, and constants. We use this grammar to employ a grammatical evolution (GE) engine to optimize guard conditions through an iterative process - starting with an initial population of candidate expressions and using genetic operators like crossover and mutation to refine them based on their classification accuracy. Finally, the best-performing guard condition is selected and represented as a function of learned coefficients and a threshold value. This methodology enables the systematic discovery of guards that accurately capture transition-enabling conditions within process models.

We use the following fitness function in this procedure:

$$\text{fit}(grd) = \sum_{\substack{t \in \text{EventLog} \\ i \leq |t|}} \begin{cases} rwd(t,i,1) & \text{if } t[..i] \models grd, \\ rwd(t,i,0) & \text{if } t[..i] \not\models grd. \end{cases}$$

As said above, the rwd function is a mechanism for plugging user Heuristics in our GP-based algorithm.

For the experiments outlined below, we used process trees, T, where each activity has a single node in the process tree that triggers it. We also restricted our experiments to guard on activity nodes only. In this case we can easily compute the set

$$enb(t, i) = \{\sigma \in \Sigma \colon \exists t' \in \Sigma^*(t[..(i-1)]\sigma t' \text{ fits } T)\}$$

of enabled activities at the ith activity in t. Using this function, we can define the reward for the guard on the activity α as follows:

$$rwd(t, i, 1) = \begin{cases} 1 & \text{if } t[i] = \alpha, \\ -1 & \text{if } t[i] \neq \alpha \text{ and } \alpha \in enb(t, i); \end{cases}$$

and $rwd(t, i, 0) = -rwd(t, i, 1)$.

IX. Experiments

The following experiments were designed to evaluate the impact of integrating guards into different process miners [25]. The experiments aim to assess the trade-offs between precision and simplicity [26] across three common mining approaches: Heuristic Miner [9], Inductive Miner [10], and ILP Miner [11].

A. Experimental Setting

This section provides details about the datasets used, the token replay conformance checking approach that we took, and the associated formulas [27].

1) Datasets: The evaluation utilized competition event logs to ensure comprehensive testing. The chosen datasets represent a wide range of real-world processes, enabling the assessment of the model's applicability across diverse scenarios.Key datasets include: BPI 2013 (BPI), Hospital Billing (HP), Order to Cash (OTC), Permit Log (PL), Travel Cost (TC), Request for Payment (RFP), Road Traffic (RT), Purchase-to-Pay (P2P).

2) Token Replay Conformance Checking: The token replay technique was employed to evaluate conformance. This approach involves converting the process models into Petri nets and analyzing their alignment with event logs. Token replay is a robust method for assessing the fitness, precision, and simplicity of process models [28], and its application ensures a detailed understanding of how well the Guarded Process Tree model aligns with observed behaviors.

B. Results

The results are divided into three experiments, comparing the performance of the Heuristic. Miner, Inductive Miner, and ILP Miner, with and without guards.

1) Heuristic Miner: The Heuristic Miner results demonstrate notable improvements in precision when guards are included, although there is a slight decrease in simplicity. This trade-off is acceptable because the primary objective of incorporating guards is to enhance precision, ensuring that the model captures observed behaviors more accurately without allowing excessive deviations. Simplicity, while important, is secondary when the goal is to improve model accuracy.

Dataset	Heuristic Miner		Heuristic Miner with Guards	
	Simplicity	Precision	Simplicity	Precision
BPI	0.474	0.748	0.468	0.776
P2P	0.569	0.719	0.224	0.790
PL	0.477	0.889	0.464	0.891
TC	0.531	0.893	0.478	0.893

Table I: Comparison of conformance for Heuristic Miner

The results in Table I highlight that, for datasets such as BPI 2013 and P2P, the addition of guards led to a measurable increase in precision, which is crucial for accurately representing the observed. For Heuristic Miner, the precision improvements across datasets are as follows: BPI: 3.74%, P2P: 9.87%, PL: 0.22%, and TC: 0.0%, with an overall average increase of 4.94%.

2) ILP Miner: The ILP Miner reveal similar trends, where the inclusion of guards improves precision across most datasets. This miner is known for producing highly precise models at the cost of simplicity, and the addition of guards amplifies this characteristic. The significant increase in precision justifies the added complexity, especially for datasets where precise behavior representation is critical.

Dataset	ILP Miner		ILP Miner with Guards	
	Simplicity	Precision	Simplicity	Precision
BPI	0.180	0.511	0.185	0.553
HB	0.129	0.171	0.130	0.187
OTC	0.461	0.726	0.231	0.843
P2P	0.136	0.523	0.122	0.661
PL	0.057	0.067	0.059	0.071
TC	0.075	0.184	0.077	0.185
RFP	0.131	0.136	0.134	0.144
RT	0.202	0.557	0.207	0.583

Table II: Comparison of conformance for ILP miner

3) Inductive Miner: Inductive Miner shows the most dramatic improvements in precision with the inclusion of guards. However, the simplicity of the models is impacted more significantly compared to the others Miners. This is expected because the Inductive Miner inherently produces simpler models, and adding guards introduces additional complexity.

Dataset	Inductive Miner		Inductive Miner with Guards	
	Simplicity	Fitness	Simplicity	Fitness
BPI	0.633	0.521	0.621	0.553
HB	0.623	0.416	0.528	0.446
OTC	0.725	0.781	0.257	0.892
P2P	0.552	0.390	0.107	0.665
PL	0.575	0.076	0.552	0.078
TC	0.619	0.097	0.522	0.111
RFP	0.586	0.339	0.420	0.346
RT	0.623	0.553	0.572	0.582

Table III: Comparison of conformance for Inductive Miner

The results in Table II indicate that, even for datasets with challenging behaviors like Order to Cash, the inclusion of guards leads to significant precision gains. For ILP Miner, the precision improvements across datasets are as follows: BPI: 8.22%, HB: 9.36%, OTC: 16.12%, P2P: 26.38%, PL: 5.97%, TC: 0.54%, RFP: 5.88%, and RT: 4.66%, with an overall average increase of 9.89%.

Table III demonstrates that, for datasets like Order to Cash and P2P, precision improvements are substantial, highlighting the ability of guards to address overgeneralization issues. For Inductive Miner, the precision improvements across datasets are as follows: BPI 2013: 6.14%, HB: 7.21%, OTC: 14.20%, P2P: 28.21%, PL: 2.63%, TC: 14.43%, RFP: 2.06%, and RT: 5.24%, with an overall average increase of 11.14%.

Across all three miners, the addition of guards consistently improves precision, with average increases of approximately 4.94% for Heuristic Miner, 11.14% for Inductive Miner, and 9.89% for ILP Miner. These results underscore the utility of guards in enhancing model precision without overgeneralizing behaviors. While simplicity is slightly reduced, the trade-off is deemed acceptable, as the enhanced precision aligns models more closely with observed event logs.

The Guarded Process Tree remains consistent and user-friendly. Since we focus on the visualization rather than the underlying Petri net, the added complexity does not negatively impact the usability or interpretation of the model in the summary findings. These findings highlight the potential for broader adoption of guarded models in process mining, particularly for applications requiring high precision and reliability.

References

[1] W. M. P. van der Aalst. Process Mining: Data Science in Action. Springer, 2nd edition, 2016.

[2] A. J. M. M. Weijters, W. M. P. van der Aalst, and A. K. Alves de Medeiros. Process mining: A 360

degree overview. In BPM (Short Papers), volume 99 of Lecture Notes in Business Information Processing, pages 1–17. Springer, 2011.

[3] A. F. Ghahfarokhi, G. Park, A. Berti, and W. M. P. van der Aalst. Ocel: A standard for object-centric event logs. In ADBIS (Short Papers), volume 1450 of Communications in Computer and Information Science, pages 90–98. Springer, 2021.

[4] B. F. van Dongen and W. M. P. van der Aalst. Detecting implicit dependencies between tasks from event logs. In Business Process Management Workshops, volume 3812 of Lecture Notes in Computer Science, pages 77–88. Springer, 2005.

[5] A.K. Alves de Medeiros. Genetic Process Mining. PhD thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2006.

[6] T. Murata. Petri nets: Properties, analysis, and applications. Proceedings of the IEEE, 77(4):541–580, 1989.

[7] Fabrizio Genovese and Jelle Herold. Executions in (semi-)integer petri nets are compact closed categories. In Electronic Proceedings in Theoretical Computer Science, volume 287, pages 127–144, 2019.

[8] J. Carmona, B. F. van Dongen, A. Solti, and M. Weidlich. Conformance Checking: Relating Processes and Models. The Information Systems Series. Springer, 2018.

[9] A.J.M.M. Weijters, W.M.P. van der Aalst, and A.K. Alves de Medeiros. Process mining with the heuristicsminer algorithm. Technical Report WP 166, Eindhoven University of Technology, Eindhoven, The Netherlands, 2006.

[10] S. J. Leemans, D. Fahland, and W. M. van der Aalst. Discovering block-structured process models from event logs: A constructive approach. In Applications and Theory of Petri Nets, pages 311–329. Springer, 2013.

[11] J. van der Werf, B. van Dongen, C. Hurkens, and A. Serebrenik. Process discovery using integer linear programming. In Applications and Theory of Petri Nets, pages 368–387. Springer, 2008.

[12] Walid Belkhir, Yannick Chevalier, and Michael Rusinowitch. Guarded variable automata over infinite alphabets. arXiv preprint arXiv:1304.6297, 2013.

[13] Vlad Rusu and Elena Zinovieva. Analyzing automata with presburger arithmetic and uninterpreted function symbols. Electronic Notes in Theoretical Computer Science, 50(4):327–341, 2001.

[14] Kurt Jensen. A brief introduction to coloured petri nets. Lecture Notes in Computer Science, 1215:203–208, 1997.

[15] Sangita Kansal and Payal Dabas. An introduction to signed petri net. Journal of Mathematics, 2021:1–8, 2021.

[16] Maja Pesic and Wil M.P. van der Aalst. Declare: Full support for loosely-structured processes. In 11th IEEE International Enterprise Distributed Object Computing Conference (EDOC 2007), pages 287–300. IEEE, 2007.

[17] Fabrizio Maria Maggi, Marco Montali, Michael Westergaard, and Wil M.P. van der Aalst. Monitoring business constraints with the event calculus. In ACM Transactions on Intelligent Systems and Technology (TIST), volume 5, pages 1–30. ACM, 2011.

[18] Chiara Di Francescomarino, Marlon Dumas, Fabrizio Maria Maggi, and Irene Teinemaa. Discovering declarative process models through cross entropy minimization. Information Systems, 53:265–283, 2015.

[19] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. Fundamentals of Business Process Management. Springer, 2013.

[20] Wil M.P. van der Aalst. Process Mining: Data Science in Action. Springer, 2016.

[21] A. F. Ghahfarokhi, A. van der Aalst, and W. M. P. van der Aalst. Translating workflow nets to process trees: An algorithmic approach. In International Conference on Business Process Management (BPM), volume 6336 of Lecture Notes in Computer Science, pages 293–310. Springer, 2010.

[22] Miguel Nicolau and Alexandros Agapitos. Understanding grammatical evolution: Grammar design. In Handbook of Grammatical Evolution, pages 23–44. Springer, 2018.

[23] Pradnya A. Vikhar. Evolutionary algorithms: A critical review and its future prospects. In Proceedings of the 2016 International Conference on Global Trends in Signal Processing, Information Computing and Communication (ICGTSPICC), pages 261–265, Jalgaon, India, 2016. IEEE.

[24] ScienceDirect. Backus-naur form. https://www.sciencedirect.com/topics/computer-science/backus-naur-form, n.d. Accessed: 2025-02-21.

[25] Wil M. P. van der Aalst. Process mining: Overview and opportunities. ACM Transactions on Management Information Systems, 3(2):7:1–7:17, 2012.

[26] Jorge Muñoz-Gama and Josep Carmona. A fresh look at precision in process conformance. In Business Process Management (BPM), volume 6336 of Lecture Notes in Computer Science, pages 211–226. Springer, 2010.

[27] A. Rozinat and W. M. P. van der Aalst. Conformance checking of processes based on monitoring real behavior. In Applications and Theory of Petri Nets, pages 425–442. Springer, 2008.

[28] Zan Huang and Akhil Kumar. A study of quality and accuracy tradeoffs in process mining. INFORMS Journal on Computing, 22(4):489–504, 2010.

Appendix A
Translation of Guarded Trees to Petri Nets

The translation is defined recursively on the structure of the guarded process tree. We start by defining the translation of the base case, which is an activity $x \in (\Sigma \cup \{\tau\})$:

Definition 10 (Base Case). If T is an activity $x \in (\Sigma \cup \{\tau\})$, then $GPN(T) = (PLC, TRN, \rightarrow, G)$ is defined as follows:

- Two places: $PLC = \{(SRC, 1), (SNK, 0)\}$.
- One transition: $TRN = \{(LEAF, x)\}$.
- The source is connected to the transition, which is connected to the sink: $\rightarrow = \{SRC \rightarrow (LEAF, x), (LEAF, x) \rightarrow SNK\}$.
- Guard: $G((LEAF, x)) = True$.

Graphically, this definition is represented in Figure 3. It shows that we wrap the activity with a source and target nodes and set the initial marking to contain a single token at the source.

Next, we define the translation of the sequential composition of guarded process trees, assuming that the translation of the sub-trees is already defined:

Definition 11. If $T =\rightarrow ((g_1, T_1), \ldots, (g_n, T_n))$ and $GPN(T_i) = (PLC_i, TRN_i, \rightarrow_i, G_i)$ for each $i = 1, \ldots, n$, we define $GPN(T) = (PLC, TRN, \rightarrow, G)$ as follows:

- Copy subnet places (excluding sources) and add global source SRC and sink SNK:
$$PLC = \{(SRC, 1), (SNK, 0)\} \cup \bigcup_{i=1}^{n} PLC_i' \setminus \{((n, SNK), 0)\}.$$
where $PLC_i' = \{((i, p), m) \mid (p, m) \in PLC_i, p \neq SRC\}$.
- Copy subnet transitions:
$$TRN = \bigcup_{i=1}^{n} \{((i, l), x) \mid (l, x) \in TRN_i\}.$$
- Redirect source edges from the first subnet to SRC, and for $i > 1$, from the previous subnet's sink:
$$\frac{SRC \xrightarrow{\lambda}_1 (l, x)}{SRC \xrightarrow{\lambda} ((1, l), x)} \quad \frac{SRC \xrightarrow{\lambda}_i (l, x)}{(i-1, SNK) \xrightarrow{\lambda} ((i, l), x)} \quad \frac{(l, x) \xrightarrow{\lambda}_n SNK}{((n, l), x) \xrightarrow{\lambda} SNK}.$$
- Other internal edges are copied:
$$\frac{n_1 \rightarrow_i n_2, \quad n_1 \neq SRC, \quad n_2 \neq SNK}{(i, n_1) \xrightarrow{\lambda} (i, n_2)}.$$
- Adjust guards:
$$G(t) = \begin{cases} G_i(t) \wedge g_i & \text{if } SRC \xrightarrow{\lambda} t \text{ for some } \lambda; \\ G_i(t) & \text{otherwise.} \end{cases}$$

The idea behind the translation is to create a new source and sink for the entire process and to connect the sub-processes in a way that ensures that the transitions are fired in the correct order. The guards are adjusted to ensure that the transitions are only enabled when the correct activity is observed in the trace. This is illustrated in Figure 3.

We define the translation of the xor composition of guarded process trees. The translation is illustrated in Figure 3. The idea behind the translation is to create a new source and sink for the entire process and to connect the sub-processes in a way that ensures that only one of the transitions is fired at a time. The guards are attached to ensure that the transitions are enabled when the sub-tree is enables.

Definition 12. If $T = \times((g_1, T_1), \ldots, (g_n, T_n))$ and $GPN(T_i) = (PLC_i, TRN_i, \rightarrow_i, G_i)$ for each $i = 1, \ldots, n$, we define $GPN(T) = (PLC, TRN, \rightarrow, G)$ as follows:

- Copy subnet places (excluding sources and sinks) and add global source SRC and sink SNK:
$$PLC = \{(SRC, 1), (SNK, 0)\} \cup \bigcup_{i=1}^{n} PLC_i',$$
where
$$PLC_i' = \{((i, p), m) \mid (p, m) \in PLC_i, \ p \neq SRC, \ p \neq SNK\}.$$
- Copy subnet transitions:
$$TRN = \bigcup_{i=1}^{n} \{((i, l), x) \mid (l, x) \in TRN_i\}.$$
- Redirect source edges to the global SRC, and sink edges to the global SNK:
$$\frac{SRC \xrightarrow{\lambda}_i (l, x)}{SRC \xrightarrow{\lambda} ((i, l), x)} \quad \frac{(l, x) \xrightarrow{\lambda}_i SNK}{((i, l), x) \xrightarrow{\lambda} SNK}.$$
- Other internal edges are copied:
$$\frac{n_1 \rightarrow_i n_2, \quad n_1 \neq SRC, \quad n_2 \neq SNK}{(i, n_1) \xrightarrow{\lambda} (i, n_2)}.$$
- Adjust guards: As defined in Definition 11.

Next, we define the translation of the parallel composition operator, which is defined as follows:

Definition 13. If $T = +((g_1, T_1), \ldots, (g_n, T_n))$ and $GPN(T_i) = (PLC_i, TRN_i, \rightarrow_i, G_i)$ for each $i = 1, \ldots, n$, we define $GPN(T) = (PLC, TRN, \rightarrow, G)$ as follows:

- Copy all subnet places and add global source SRC and sink SNK:
$$PLC = \{(SRC, 1), (SNK, 0)\} \cup \bigcup_{i=1}^{n} PLC_i',$$
where
$$PLC_i' = \{((i, p), m) \mid (p, m) \in PLC_i\}.$$
- Copy all subnet transitions and add two global τ-transitions:
$$TRN = \bigcup_{i=1}^{n} \{((i, l), x) \mid (l, x) \in TRN_i\} \cup \{(PRE, \tau), (POST, \tau)\}.$$
- Add synchronization edges: from SRC to PRE, from PRE to each (i, SRC), from each (i, SNK) to $POST$, and from $POST$ to SNK:
$$SRC \rightarrow PRE, \quad \forall i : PRE \rightarrow (i, SRC), \quad (i, SNK) \rightarrow POST, \quad POST$$
- Other edges are copied:
$$\frac{n_1 \rightarrow_i n_2}{(i, n_1) \xrightarrow{\lambda} (i, n_2)}.$$
- Adjust guards: As defined in Definition 11.

As illustrated in Figure 3, the translation of the parallel composition operator involves creating new nodes and activities, and connecting them to the sub-processes to ensure that all sub-trees run simultaneously. The guards are adjusted to ensure that the initial transitions are enabled when the sub-tree is enabled, preserving the semantics of the original Guarded Process Tree.

Finally, we define the translation of the loop composition operator, which is defined as follows:

Definition 14. If $T = *((g_1, T_1), (g_2, T_2))$ and $GPN(T_i) = (PLC_i, TRN_i, \to_i, G_i)$ for $i = 1, 2$, we define $GPN(T) = (PLC, TRN, \to, G)$ as follows:

- Copy subnet places (excluding sources and sinks), and add global source SRC and sink SNK:

$$PLC = \{(SRC, 1), (SNK, 0)\} \cup \bigcup_{i=1}^{2} PLC'_i,$$

where

$$PLC'_i = \{((i,p), m) \mid (p,m) \in PLC_i,\ p \neq SRC,\ p \neq SNK\}.$$

- Copy all subnet transitions:

$$TRN = \bigcup_{i=1}^{2} \{((i,l),x) \mid (l,x) \in TRN_i\}.$$

- Redirect edges:

$$\frac{SRC \xrightarrow{\lambda}_1 (l,x)}{SRC \xrightarrow{\lambda} ((1,l),x)}, \quad \frac{(l,x) \xrightarrow{\lambda}_1 SNK}{((1,l),x) \xrightarrow{\lambda} SNK},$$

$$\frac{SRC \xrightarrow{\lambda}_2 (l,x)}{SNK \xrightarrow{\lambda} ((2,l),x)}, \quad \frac{(l,x) \xrightarrow{\lambda}_2 SNK}{((2,l),x) \xrightarrow{\lambda} SRC}.$$

- Other internal edges are copied:

$$\frac{n_1 \to_i n_2, \quad n_1 \neq SRC, \quad n_2 \neq SNK}{(i,n_1) \xrightarrow{\lambda} (i,n_2)}.$$

- Adjust guards: As defined in Definition 11.

The translation of the loop composition operator is illustrated in Figure 3. The idea behind the translation is to create a new source and sink for the entire process and to connect the sub-processes in a way that ensures that the sub-trees are executed in a loop.

Appendix B
Proof of Proposition 1

Proposition 2. A sequence $(x_1, \ldots, x_l)$ is a trace of the Guarded-Petri-Net $GPN = (PLC, TRN, \to, G)$ if and only if it is a trace of the Petri-Net $PN(GPN) = (PLC', TRN', \to')$.

Proof. Let the weight function $W(s,d)$ define the weight of an arc between two elements s and d in the Guarded Petri Net, where s and d can be places or transitions:

$$W(s,d) = \begin{cases} w & \text{if } (s,w,d) \in \to \\ 0 & \text{otherwise} \end{cases}.$$

Let the weight function $W'(s,d)$ define the weight of an arc between two elements s and d in the Petri Net, where s and d can be places or transitions:

$$W'(s,d) = \begin{cases} w & \text{if } (s,w,d) \in \to' \\ 0 & \text{otherwise} \end{cases}.$$

Let

$$\phi_{t,i,j} = \sum_{k=0}^{n_{t,i,j}} \alpha_{t,i,j,k} x_{t,i,j,k} \geq C_{t,i,j}$$

represent a single numeric constraint for the j-th condition in the i-th group of conditions for a transition t and

$$\phi_{t,i} = \bigwedge_{j=1}^{n_{t,i}} \left(\sum_{k=0}^{n_{t,i,j}} \alpha_{t,i,j,k} x_{t,i,j,k} \geq C_{t,i,j} \right)$$

represent the combined conjunction of all constraints indexed with j for the i-th group of conditions for a transition t.

($\Rightarrow$):

- Let $(M_0, t_0, \cdots, t_{n-1}, M_n)$ be a run of GPN for a trace $(x_1, \ldots, x_n)$.
- Let $M'_r \colon PLC' \to \mathbb{Z}$ be defined by

$$M'_r(p) = \begin{cases} M_r(p) & \text{if } p \in PLC, \\ 1 - C_{t,i,j} + ctr(t,i,j) & \text{if } p = p_{t,i,j} \text{ for some } t,i,j. \end{cases}$$

where $ctr(t,i,j,k) = \alpha_{t,i,j,k} * \left|\{s \leq r \colon x_s = x_{t,i,j,k}\}\right|$ and $ctr(t,i,j) = \sum_{k=0}^{n_{t,i,j}} ctr(t,i,j,k)$.
- We show for each $i \in [0, n-1]$ there is $t'_i = (l_i, x_i) \in \to'$ such that $(M'_0, t'_0, \cdots, t'_{n-1}, M'_n)$ is a run of $PN(GPN)$ with the trace $(x_1, \ldots, x_n)$.
  1) Assume, by induction, that $(M'_0, t'_0, \cdots, t'_{r-2} M'_{r-1})$ is a run of $PN(GPN)$ with the trace $(x_1, \ldots, x_{r-1})$
  2) Since $(M_0, t_0, \cdots, t_{n-1}, M_n)$ is a run of GPN , $t = (l_r, x_r)$ enabled at M_{r-1} after reading $(x_1, \ldots, x_{r-1})$.
  3) Since t can fire in $PN(GPN)$, $G((l_r, x_r))(x_1, \ldots, x_{r-1}) =$ True. So there is i such $\phi_{t,i} =$ True , thus $\forall j \in [n_{t,i}]$, $\phi_{t,i,j} =$ True.
  4) Define $t' = ((i, l_r), x_r) \in TRN'$ and show t' is enabled at M'_{r-1}:
     a) For $p \in PLC$: Since t is enabled at M_{r-1}, $M_{r-1}(p) \geq W(p,t)$.
     Thus, $M'_{r-1}(p) = M_{r-1}(p) \geq W'(p,t')$.
     b) For $p = p_{t,i,j} \in PLC' \setminus PLC$: By Definition 9 (section 3.b) and Definition 9 (section 3.c),

$$M'_{r-1}(p) = 1 - C_{t,i,j} + ctr(t,i,j).$$

Since $\phi_{t,i,j}$ is satisfied,

$$ctr(t,i,j) \geq C_{t,i,j}.$$

Hence,

$$M'_{r-1}(p) = 1 - C_{t,i,j} + ctr(t,i,j) \geq 1.$$

Thus, $M'_{r-1}(p) \geq W'(p,t')$.

5) Marking Update correctness:
   a) For $p \in PLC$: $M'_r(p) = M_r(p) = M_{r-1}(p) + W(t,p) - W(p,t) = M'_{r-1}(p) + W'(t',p) - W'(p,t')$.
   b) For $p = p_{t,i,j} \in PLC' \setminus PLC$: $M'_r(p) = 1 - C_{t,i,j} + ctr(t,i,j) = M'_{r-1}(p) + W'(t',p) - W'(p,t')$.
6) At the final marking, $M'_n(SNK) = M_n(SNK) = 1$.

Therefore, $(x_1, \ldots, x_n)$ is a trace of $PN(GPN)$.
($\Leftarrow$):

- Let $(M'_0, t'_0, \cdots, t'_{n-1}, M'_n)$ be a run of $PN(GPN)$ for a trace $(x_1, \ldots, x_n)$.
- Define $M_r \colon PLC \to \mathbb{Z}$ by $M_r(p) = M'_r(p)$.
- We show for each $i \in [0, n-1]$ there is $t_i = (l_i, x_i) \in \to$ such that $(M_0, t_0, \cdots, t_{n-1}, M_n)$ is a run of GPN with the trace $(x_1, \ldots, x_l)$:
  1) Assume, by induction, that $(M_0, t_0, \cdots, t_{r-2}, M_{r-1})$ is a run of GPN with the trace $(x_1, \ldots, x_{r-1})$.
  2) Since $(M'_0, t'_0, \cdots, t'_{n-1}, M'_n)$ be a run of $PN(GPN)$, $t' = ((i, l_r), x_r)$ enabled at M'_{r-1}, so define $t = (l_r, x_r)$ and show it is enabled at M_{r-1}: Denote $p \in PLC$.
     a) Since t' is enabled at M'_{r-1}, $M'_{r-1}(p) \geq W'(p,t')$. Using Definition 9 (section 3.a), $W(p,t) = W'(p,t')$. Thus, $M_{r-1}(p) = M'_{r-1}(p) \geq W(p,t)$.
     b) Since t' can fire in $PN(GPN)$, there is i such $p_{t,i,j}$ contains at least one token $\forall j \in \{1, \ldots, n_{t,i}\}$.
     By definition Definition 9 (section 3.b) and Definition 9 (section 3.c),

$$M'_{r-1}(p) = 1 - C_{t,i,j} + ctr(t,i,j) \geq 1.$$

     thus,

$$ctr(t,i,j) \geq C_{t,i,j}.$$

     where $ctr(t,i,j,k) = \alpha_{t,i,j,k} * \left|\{s \leq r \colon x_s = x_{t,i,j,k}\}\right|$ and $ctr(t,i,j) = \sum_{k=0}^{n_{t,i,j}} ctr(t,i,j,k)$. Hence, $\phi_{t,i,j}$ is true. Thus, $\phi_{t,i}$ is true implying $G((l_r, x_r))(x_1, \ldots, x_{r-1}) = \text{True}$.
  3) Marking Update correctness:
  4) For $p \in PLC$: $M_r(p) = M'_r(p) = M'_{r-1}(p) + W'(t',p) - W'(p,t') = M_{r-1}(p) + W(t,p) - W(p,t)$.
  5) At the final marking, $M_n(SNK) = M'_n(SNK) = 1$.

Therefore, $(x_1, \ldots, x_n)$ is a trace of $GPN = (PLC, TRN, \to, G)$. □